

DATABASE IMPLEMENTATION PLAN

Clothing E-Commerce Platform — Core Assignment Version

This database plan is aligned with the finalized **Customer/Admin Frontend, API Gateway, Auth Service, User Service, Product Service, Inventory Service, Order Service, Payment Service, Notification Service**, JWT authentication, Cloudinary, Stripe Test Mode, RabbitMQ, Redis, and later Docker Compose integration.

1. Database Goal

The database stores all persistent business data required by the clothing e-commerce platform.

We will use

```
PostgreSQL
  ↓
Prisma ORM
  ↓
Node.js / Express Backend
```

During development

```
PostgreSQL
  ↑↓
pgAdmin 4
```

Later

```
Docker Compose
  ↓
PostgreSQL Container
```

The database can therefore be designed and tested locally using PostgreSQL + pgAdmin 4 before Docker integration.

2. Database Technology Stack

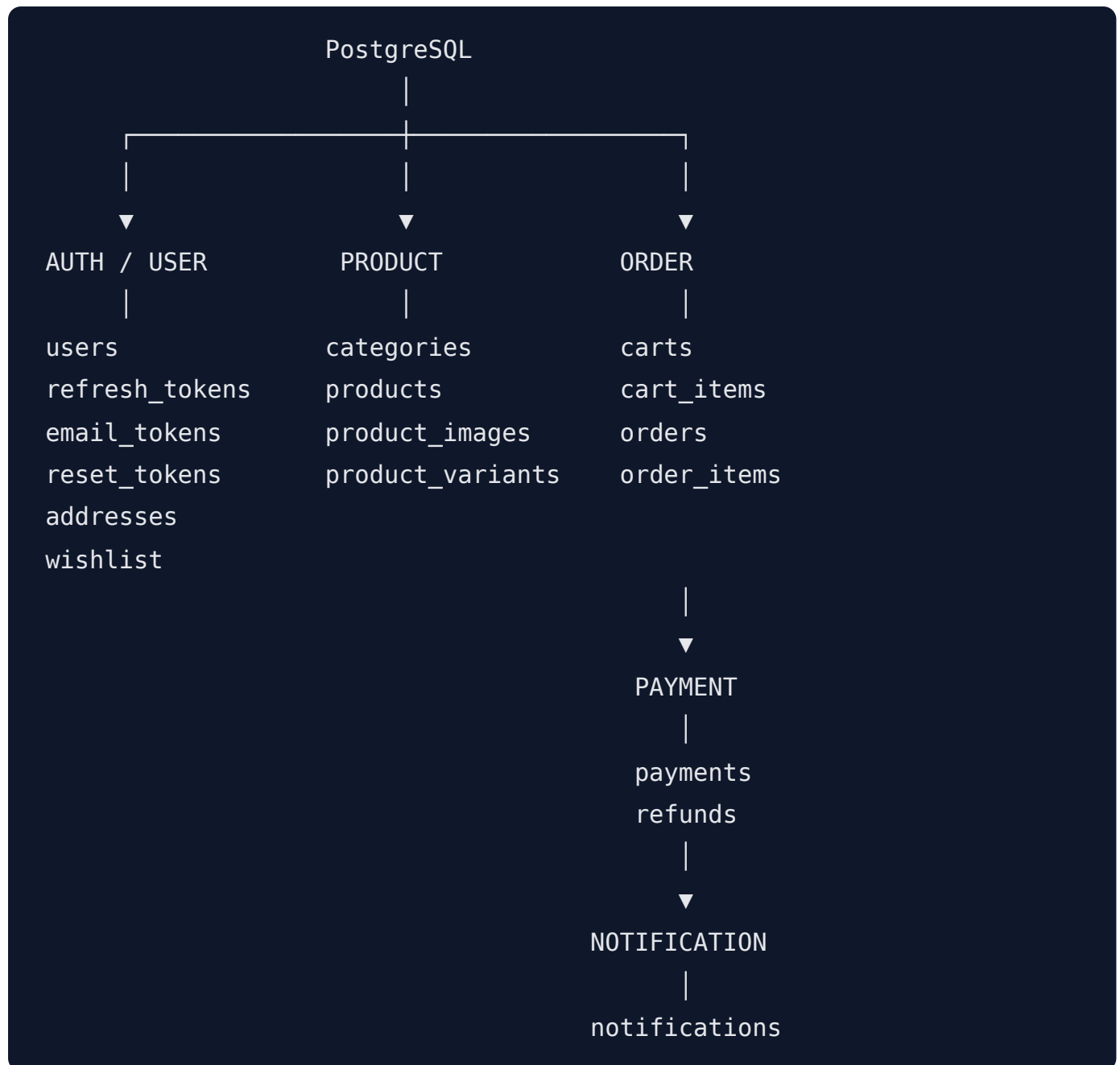
Purpose	Technology
Main Database	PostgreSQL
Database GUI	pgAdmin 4
ORM	Prisma ORM
Schema	Prisma Schema
Migrations	Prisma Migrate
Seed Data	Prisma Seed
Primary Keys	UUID
Money	DECIMAL(10,2)
Cache	Redis
Message Broker	RabbitMQ
Product Images	Cloudinary URLs
Payment Gateway	Stripe Test Mode
Containerization	Docker Compose (Later)

Redis and RabbitMQ are infrastructure services.

They do **not** require PostgreSQL tables.

3. Database Architecture

For the assignment use **one PostgreSQL database**.



The backend is divided into multiple microservices, but all services share a single PostgreSQL database.

4. Final Tables

The database contains **17 core tables**.

AUTH

1. users
2. refresh_tokens

3. email_verification_tokens

4. password_reset_tokens

USER

5. addresses

6. wishlist_items

PRODUCT

7. categories

8. products

9. product_images

10. product_variants

ORDER

11. carts

12. cart_items

13. orders

14. order_items

PAYMENT

15. payments

16. refunds

NOTIFICATION

17. notifications

No Review table is included in the core assignment.

5. Users Table

users

Stores customer and admin accounts.

Column	Type	Rules
id	UUID	PK
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	UNIQUE
phone	VARCHAR(20)	NULL
password_hash	TEXT	NOT NULL
role	UserRole	DEFAULT CUSTOMER
email_verified	BOOLEAN	DEFAULT FALSE
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	Prisma @updatedAt

Roles

CUSTOMER

ADMIN

Passwords are always stored as hashes.

Never

```
password = abc123
```

Always

```
password_hash = $2b$...
```

6. Refresh Tokens

refresh_tokens

Stores JWT refresh tokens.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK
token_hash	TEXT	UNIQUE
expires_at	TIMESTAMP	NOT NULL
revoked_at	TIMESTAMP	NULL
created_at	TIMESTAMP	DEFAULT NOW()

Relationship



Supports multiple devices.

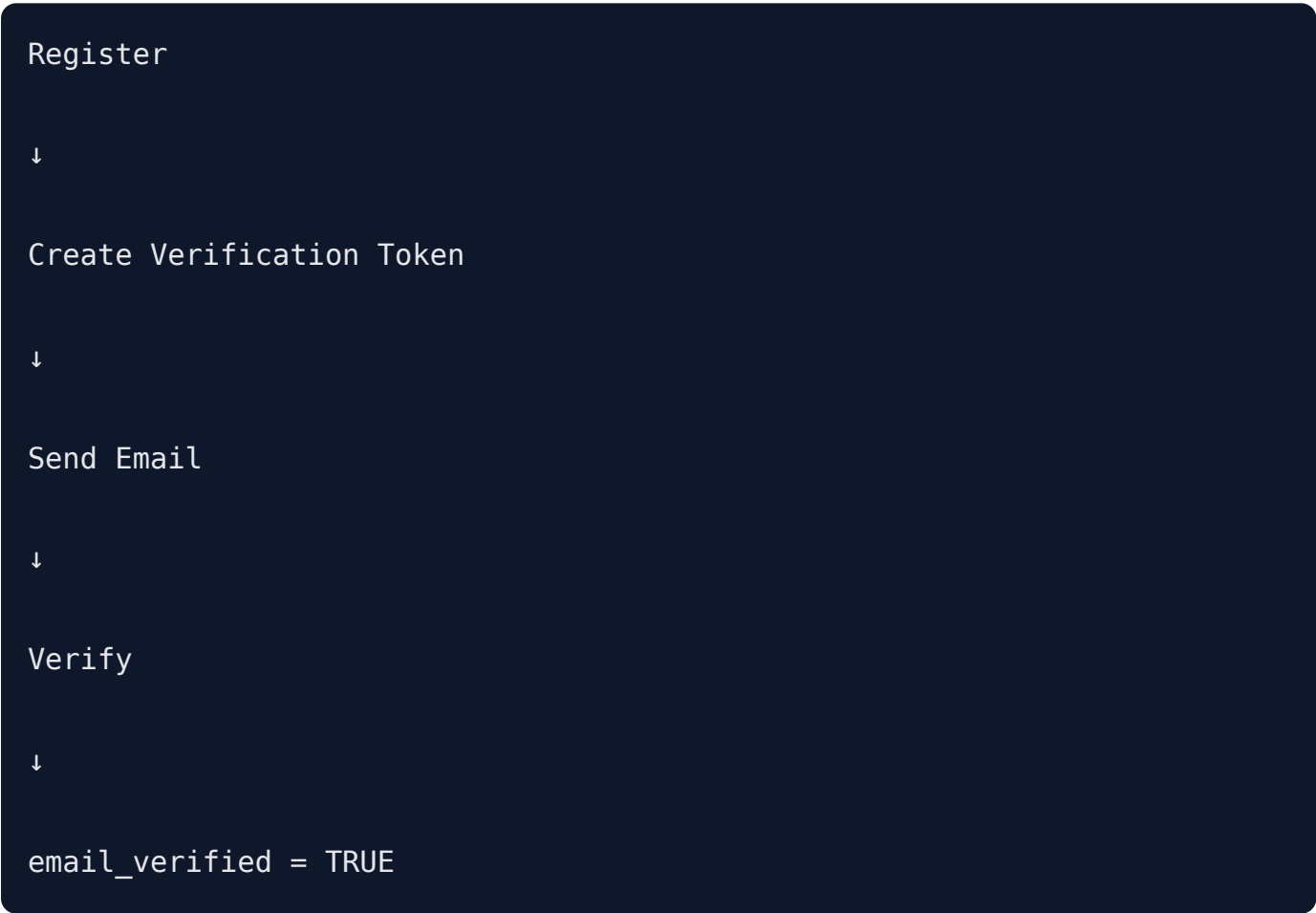
7. Email Verification Tokens

email_verification_tokens

Used for email verification.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK
token_hash	TEXT	UNIQUE
expires_at	TIMESTAMP	NOT NULL
used_at	TIMESTAMP	NULL
created_at	TIMESTAMP	DEFAULT NOW()

Flow



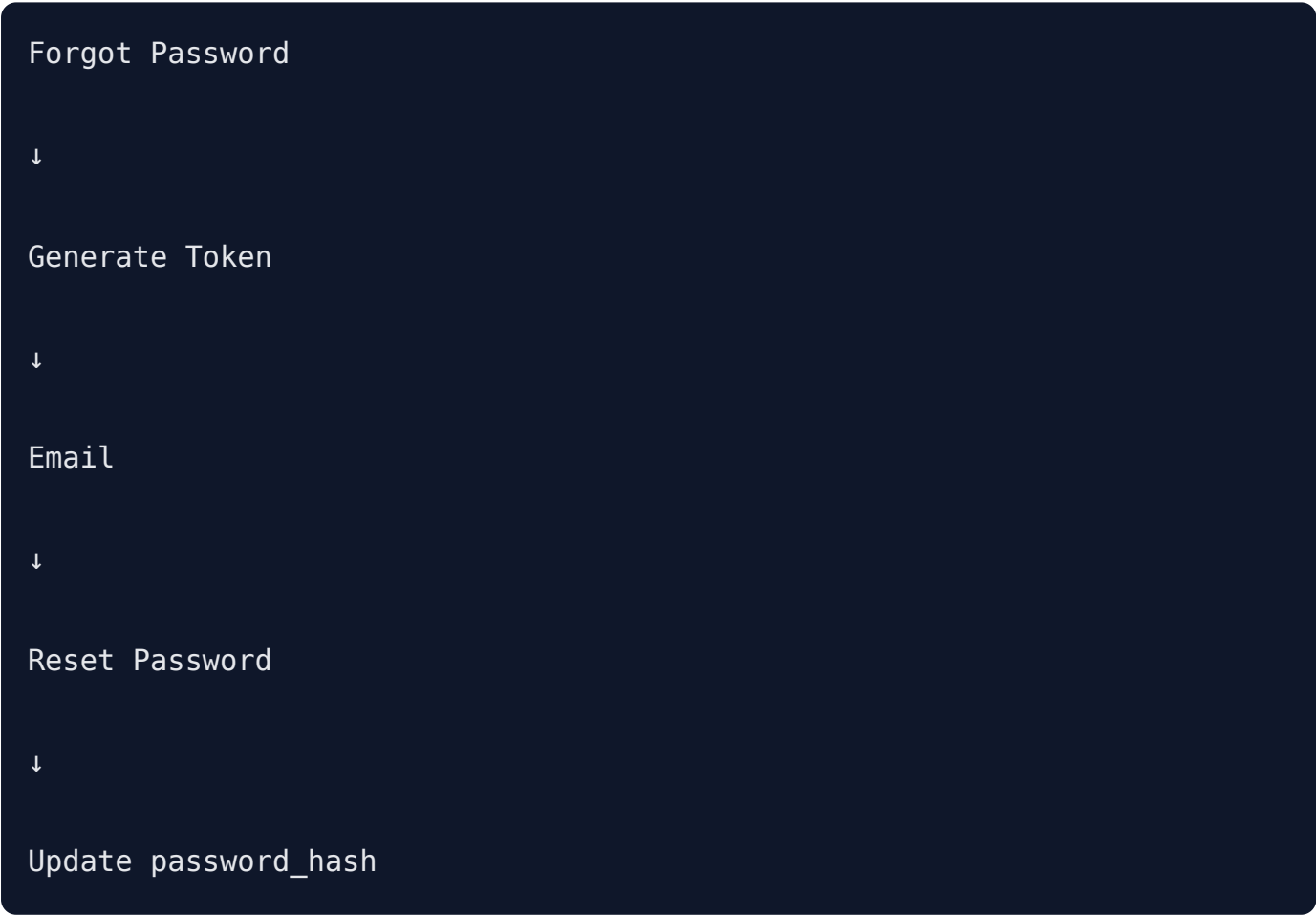
8. Password Reset Tokens

password_reset_tokens

Used for forgot password.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK
token_hash	TEXT	UNIQUE
expires_at	TIMESTAMP	NOT NULL
used_at	TIMESTAMP	NULL
created_at	TIMESTAMP	DEFAULT NOW()

Flow



9. Addresses

addresses

Stores delivery addresses.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK
full_name	VARCHAR(150)	NOT NULL
phone	VARCHAR(20)	NOT NULL
address_line1	VARCHAR(255)	NOT NULL
address_line2	VARCHAR(255)	NULL
city	VARCHAR(100)	NOT NULL
state	VARCHAR(100)	NOT NULL
postal_code	VARCHAR(20)	NOT NULL
country	VARCHAR(100)	DEFAULT India
is_default	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	Prisma @updatedAt

Relationship



One user can save multiple addresses.

10. Categories

categories

Stores clothing categories.

Column	Type	Rules
id	UUID	PK
name	VARCHAR(100)	UNIQUE
slug	VARCHAR(120)	UNIQUE
description	TEXT	NULL
image_url	TEXT	NULL
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	Prisma @updatedAt

Examples

T-Shirts

Shirts

Hoodies

Jeans

Joggers

Jackets

Accessories

11. Products

products

Stores the main catalog.

Column	Type	Rules
id	UUID	PK
category_id	UUID	FK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(280)	UNIQUE
description	TEXT	NOT NULL
audience	ProductAudience	NOT NULL
price	DECIMAL(10,2)	NOT NULL
discount_price	DECIMAL(10,2)	NULL
is_active	BOOLEAN	DEFAULT TRUE

created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	Prisma @updatedAt

Audience

```
MEN

WOMEN

UNISEX
```

Supports frontend navigation

```
Men

Women

Categories
```

Constraints

```
price >= 0

discount_price >= 0

discount_price <= price
```

12. Product Images

product_images

Stores Cloudinary image references.

Column	Type	Rules
--------	------	-------

id	UUID	PK
product_id	UUID	FK
image_url	TEXT	NOT NULL
public_id	VARCHAR(255)	NOT NULL
is_primary	BOOLEAN	DEFAULT FALSE
display_order	INTEGER	DEFAULT 0
created_at	TIMESTAMP	DEFAULT NOW()

Flow



Only URLs are stored.

13. Product Variants (Inventory)

product_variants

Stores

- SKU

- Size
- Color
- Inventory

No separate inventory table is required.

Column	Type	Rules
id	UUID	PK
product_id	UUID	FK
sku	VARCHAR(100)	UNIQUE
size	VARCHAR(20)	NOT NULL
color	VARCHAR(50)	NOT NULL
stock_quantity	INTEGER	DEFAULT 0
reserved_quantity	INTEGER	DEFAULT 0
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	Prisma @updatedAt

Constraints

```
stock_quantity >= 0

reserved_quantity >= 0

reserved_quantity <= stock_quantity
```

Unique

```
UNIQUE(product_id,size,color)
```

Available Stock

```
available =  
stock_quantity  
-  
reserved_quantity
```

14. Wishlist

wishlist_items

Stores saved products.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK
product_id	UUID	FK
created_at	TIMESTAMP	DEFAULT NOW()

Unique

```
UNIQUE(user_id,product_id)
```

No duplicate wishlist entries.

15. Carts

carts

Stores customer shopping carts.

Column	Type	Rules
id	UUID	PK

user_id	UUID	FK
status	CartStatus	DEFAULT ACTIVE
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	Prisma @updatedAt

Status

ACTIVE

CONVERTED

ABANDONED

Only one ACTIVE cart should exist for a customer.

16. Cart Items

cart_items

Stores items inside carts.

Column	Type	Rules
id	UUID	PK
cart_id	UUID	FK
variant_id	UUID	FK
quantity	INTEGER	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	Prisma @updatedAt

Constraint


```
quantity > 0
```

Unique

```
UNIQUE(cart_id,variant_id)
```

17. Orders

orders

Stores customer orders.

(Continue exactly as in your uploaded structure with the same columns, using `updated_at` as Prisma `@updatedAt`.)

Order Status

```
PENDING  
CONFIRMED  
PACKED  
SHIPPED  
DELIVERED  
CANCELLED  
REFUNDED
```

Payment Status

```
PENDING  
PAID  
FAILED  
REFUNDED
```

18. Why Shipping Address Is Stored Again

Keep the same explanation from your uploaded plan.

19. Order Items

Keep the same table structure and snapshot explanation from your uploaded plan.

20. Payments

Keep the same structure.

Important:

Never store

Card Number

CVV

PIN

Only store Stripe identifiers and payment status.

21. Refunds

Keep the same structure.

Support

PENDING

SUCCESS

FAILED

22. Notifications

Keep the same structure.

Notification Types

```
ORDER_CONFIRMED
```

```
PAYMENT_CONFIRMED
```

```
PAYMENT_FAILED
```

```
ORDER_SHIPPED
```

```
ORDER_DELIVERED
```

```
REFUND_COMPLETED
```

23. Final Enum Definitions

UserRole

```
CUSTOMER
```

```
ADMIN
```

ProductAudience

```
MEN
```

```
WOMEN
```

```
UNISEX
```

CartStatus

ACTIVE
CONVERTED
ABANDONED

OrderStatus

PENDING
CONFIRMED
PACKED
SHIPPED
DELIVERED
CANCELLED
REFUNDED

OrderPaymentStatus

PENDING
PAID
FAILED
REFUNDED

PaymentStatus

PENDING
SUCCESS
FAILED
REFUNDED

RefundStatus

PENDING
SUCCESS
FAILED

NotificationType

ORDER_CONFIRMED
PAYMENT_CONFIRMED
PAYMENT_FAILED
ORDER_SHIPPED
ORDER_DELIVERED
REFUND_COMPLETED

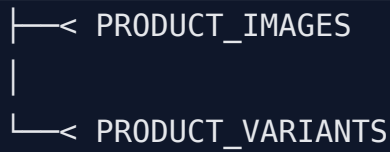
24. Complete ER Relationship Structure

USERS



CATEGORIES





25. Service → Database Mapping

This mapping is aligned with the finalized backend architecture.

Backend Service	Database Tables
Auth Service	users, refresh_tokens, email_verification_tokens, password_reset_tokens
User Service	users, addresses, wishlist_items
Product Service	categories, products, product_images
Inventory Service	product_variants
Order Service	carts, cart_items, orders, order_items
Payment Service	payments, refunds
Notification Service	notifications

The API Gateway does not require any business-data tables.

26. Frontend → Database Mapping

Frontend Feature	Database
Register/Login	users
JWT Refresh	refresh_tokens
Email Verification	email_verification_tokens

Forgot Password	password_reset_tokens
Profile	users
Addresses	addresses
Men/Women	products.audience
Categories	categories
Products	products
Product Images	product_images
Size / Color	product_variants
Stock	product_variants
Wishlist	wishlist_items
Cart	carts + cart_items
Checkout	carts + orders
Order Success	orders + order_items
My Orders	orders
Tracking	order status + timestamps
Stripe Payment	payments
Refund	refunds
Admin Products	products
Admin Categories	categories
Admin Inventory	product_variants
Admin Orders	orders
Admin Payments	payments + refunds
Dashboard Analytics	products + orders + payments

This confirms that the database fully supports the finalized frontend implementation.

27. Important Constraints

Users

```
email UNIQUE  
email NOT NULL
```

Products

```
price >= 0  
  
discount_price >= 0  
  
discount_price <= price
```

Product Variants

```
stock_quantity >= 0  
  
reserved_quantity >= 0  
  
reserved_quantity <= stock_quantity  
  
UNIQUE(product_id,size,color)
```

Wishlist

```
UNIQUE(user_id,product_id)
```

Cart Items

```
quantity > 0
```

```
UNIQUE(cart_id,variant_id)
```

Order Items

```
quantity > 0
```

```
unit_price >= 0
```

```
line_total >= 0
```

Payments / Refunds

```
amount >= 0
```

28. Important Indexes

Use only the indexes required for good query performance.

Users

```
email
```

Products

category_id

audience

name

is_active

Product Variants

product_id

sku

Addresses

user_id

Wishlist

user_id

Carts

user_id

status

Orders

user_id

status

created_at

Order Items

order_id

Payments

order_id

status

Notifications

user_id

is_read

Primary keys and UNIQUE columns already create indexes automatically.

29. Delete Strategy

Avoid deleting historical business data.

Example

```
Admin removes product
```

↓

```
products.is_active = FALSE
```

instead of

```
DELETE FROM products
```

This preserves order history.

Use cascade deletion only where appropriate.

Example

```
Product
```

↓

```
Product Images
```

and

```
Cart
```

↓

```
Cart Items
```

Orders, payments and refunds should normally never be deleted.

30. Basic Analytics

No analytics table is required.

The admin dashboard can calculate information from existing tables.

Total Products

```
COUNT(products)
```

Total Orders

```
COUNT(orders)
```

Total Revenue

```
SUM(successful_payments)
```

Low Stock Products

```
product_variants  
  
WHERE  
  
(stock_quantity - reserved_quantity) <= threshold
```

Revenue Chart

Aggregate successful payments by date.

No separate analytics database or warehouse is needed.

31. Search and Filters

The Product Service performs product search directly from PostgreSQL.

Users can search products using:

Product Name

Category

Basic Keywords

Filtering is supported using PostgreSQL queries.

Available filters:

Category

Audience

Size

Color

Price Range

Stock Availability

PostgreSQL remains the single source of truth for product search.

Redis may cache frequently accessed product lists and search results.

No separate search engine is required for this assignment.

32. Cloudinary

Product image flow

Admin



Upload Product Image



Cloudinary



Returns

image_url

public_id



product_images

Only URLs and metadata are stored inside PostgreSQL.

33. Redis

Redis is used as a caching layer.

Example

GET Products



Check Redis



HIT

MISS

|
Return

|
PostgreSQL

|
Cache Result

Redis is used for:

- Product List
- Category List

- Frequently Accessed Products
- Search Results

Redis stores temporary data only.

PostgreSQL remains the source of truth.

34. RabbitMQ

RabbitMQ enables asynchronous communication between backend services.

Example

```
Order Service
  ↓
OrderPlaced
  ↓
RabbitMQ
  ↓
Notification Service
```

Core events

```
UserRegistered

OrderPlaced

PaymentConfirmed

PaymentFailed

ShipmentUpdated

RefundCompleted
```

RabbitMQ exchanges, queues and DLQ are configured during backend/Docker integration.

No RabbitMQ table is required inside PostgreSQL.

35. Prisma Structure

Use one centralized Prisma setup.

```
database/  
|  
└─ prisma/  
    ├── schema.prisma  
    ├── migrations/  
    └─ seed.ts
```

Flow

```
Backend Repository  
    ↓  
Prisma Client  
    ↓  
PostgreSQL
```

The schema contains:

```
Enums  
  
Models  
  
Relationships  
  
Indexes  
  
Constraints
```

36. Database Folder Structure

```
database/
|
├─ prisma/
|   ├─ schema.prisma
|   ├─ migrations/
|   └─ seed.ts
|
├─ sql/
|   ├─ init.sql
|   └─ indexes.sql
|
├─ diagrams/
|   ├─ er-diagram.png
|   └─ relationships.md
|
└─ README.md
```

Later, Docker Compose will integrate the PostgreSQL database.

37. Environment Variables

```
DATABASE_URL=
```

```
postgresql://USER:PASSWORD@HOST:PORT/DATABASE
```

Never commit real credentials.

Repository

```
.env.example
```

Local Machine

```
.env
```

`.env` must be added to `.gitignore`.

38. Seed Data

Create enough data for frontend development.

Users

1 Admin

3–5 Customers

Categories

T-Shirts

Shirts

Hoodies

Jeans

Joggers

Jackets

Accessories

Products

20–40 Demo Products

Each product includes

- Category
 - Audience
 - Images
 - Variants
 - Price
 - Stock
-

Orders

5–10 Demo Orders

Use test data only.

39. Database Implementation Order

1. Install PostgreSQL + pgAdmin

↓

2. Create Database

↓

3. Create ER Diagram

↓

4. Finalize Enums

↓

5. Users/Auth Tables

↓

6. Product Tables

↓

7. Wishlist

↓

8. Cart Tables

↓

9. Order Tables

↓

10. Payment Tables

↓

11. Notification Table

↓

12. Primary Keys

↓

13. Foreign Keys

↓

14. Unique Constraints

↓

15. Check Constraints

↓

16. Indexes

↓

17. Test Relationships

↓

18. Prisma Schema

↓

19. Prisma Migration

↓
20. Seed Data
↓
21. Connect Backend
↓
22. CRUD Testing
↓
23. Checkout Flow Testing
↓
24. Payment Testing
↓
25. Docker Integration

40. Complete Checkout Database Flow

```
Customer
  ↓
Cart
  ↓
Cart Items
  ↓
Validate Product Variant Stock
  ↓
Create Order
  ↓
Create Order Items
  ↓
Reserve Inventory
  ↓
Create Payment
  ↓
Stripe Test Payment
  ↓
Payment Success
  ↓
RabbitMQ publishes PaymentConfirmed
  ↓
payments.status = SUCCESS
  ↓
orders.payment_status = PAID
  ↓
orders.status = CONFIRMED
  ↓
Update Inventory
  ↓
cart.status = CONVERTED
```

41. Admin Database Flow

Admin

|
|— Products
| ↓
| products
| categories
| product_images
| product_variants

|
|— Inventory
| ↓
| product_variants

|
|— Orders
| ↓
| orders
| order_items

|
|— Payments
| ↓
| payments
| refunds

|
|— Dashboard
| ↓
| Shows

- Total Products
- Total Orders
- Total Revenue
- Low Stock Products

42. What Is NOT Needed

To keep the assignment manageable, do not add:

Separate PostgreSQL database per microservice

Elasticsearch

Kafka

Warehouse management tables

AI recommendation tables

Loyalty points

Coupons / Complex Promotions

Product Reviews

Advanced Analytics Tables

RabbitMQ Tables

Redis Tables

Complex RBAC Permission Tables

Only two roles are required:

CUSTOMER

ADMIN

43. Final Database Scope

The final database consists of **one PostgreSQL database** containing **17 core tables**.

Users/Auth

- └─ users
- └─ refresh_tokens
- └─ email_verification_tokens
- └─ password_reset_tokens
- └─ addresses
- └─ wishlist_items

Products

- └─ categories
- └─ products
- └─ product_images
- └─ product_variants

Orders

- └─ carts
- └─ cart_items
- └─ orders
- └─ order_items

Payments

- └─ payments
- └─ refunds

Notifications

- └─ notifications

It supports the complete customer flow:

Register

- Login
- Browse Products
- Men/Women/Categories
- Basic PostgreSQL Search & Filters
- Product Details
- Size/Color/Stock
- Wishlist
- Cart
- Address
- Checkout
- Stripe Payment
- Order Tracking
- Refund

It also supports the complete admin flow:

Admin Login

- Dashboard
- Products
- Categories
- Cloudinary Images
- Inventory
- Orders
- Payments/Refunds
- Basic Analytics